Infinite Generation Two

— Detailed Technical Design —

Fides Thérèse

2026-10-22

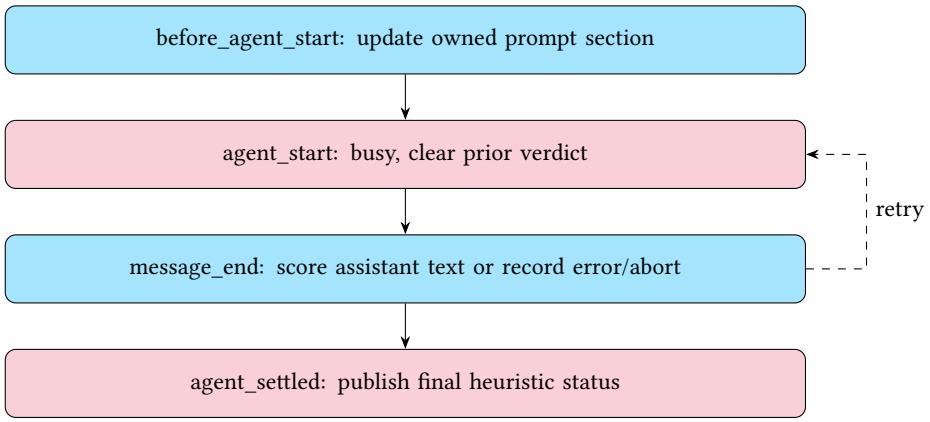
1 Document control

Status: implemented and verified against the contract below. Delivery mode: Standard. Specification: `specification.tex`. Canonical DTD: `docs/DTD.tex`. Scope: replace the DeepSeek Harness adapter with a native Pi extension while freezing the prompt. Baseline: `3a701c3`. No open product decisions or credentials are required. This document remains the implementation contract; the specification and change evidence record the executed verification.

2 Technical scope and architecture

Implement prompt-section injection, the existing parameterless profile tool, a session-local English status indicator, an enable/disable/status command, and an initial-disable flag. Preserve the MIT license and prompt exactly. Remove the browser/Cordis adapter and Harness installers, which cannot load in Pi. Do not add networking, provider selection, credential storage, background jobs, or durable state. Preserve the original response markers and matching/exclusion behavior, while accepting only valid text blocks.

The Pi process loads the adapter and executes its synchronous event handlers. The adapter depends on two leaf helpers; neither helper imports the adapter. Pi owns model calls, cancellation, approval decisions, and the session. Only Pi owns the status UI. The extension owns no timers or external resources. The specification contains the dependency diagram. The critical runtime is:



The retry arrow represents Pi's lifecycle, not a retry initiated by this package.

3 Research and technology stack

Verified on 2026-10-22 against installed Pi 0.99.1 declarations and examples, the live official Pi extension documentation and system-prompt source, and npm package metadata. npm reported Pi 1.4.0 as the current release; the compatibility floor remains the installed 0.99.1 API. Both 0.99.1 and 1.4.0 accepted the package in offline CLI loading checks; SDK/event-runner tests target 0.99.1. Newer compatibility is not inferred from version numbers. Required references:

- `https://github.com/earendil-works/pi/blob/main/packages/coding-agent/docs/extensions.md`
- `https://github.com/earendil-works/pi/blob/main/packages/coding-agent/docs/packages.md`
- `https://github.com/earendil-works/pi/blob/main/packages/coding-agent/src/core/system-prompt.ts`
- Installed `docs/sdk.md`, `docs/cli.md`, `examples/extensions/prompt-customizer.ts`, `examples/extensions/hello.ts`, and exported TypeScript declarations.

Select Node.js ≥22.19, ES modules, TypeScript 5.9.3, Node type declarations 22.19.19, Pi 0.99.1 for reproducible development, and TypeBox 1.3.27. Use Node's built-in test runner with concurrency one and native TypeScript stripping. There is no production bundler or generated JavaScript.

4 Dependency inventory

Pi coding-agent. Host-provided runtime API; declare peer range `*`, and pin 0.99.1 as a development dependency. Production imports only the types `ExtensionAPI`, `ExtensionContext`, and `ToolDefinition`. The factory receives the actual API; no duplicate host is bundled. Use `on`, `registerTool`, `registerCommand`, `registerFlag`, `getFlag`, and `sendMessage`. Pi reports load/handler errors. Do not introduce a provider implementation.

TypeBox. Host-provided schema builder; peer `*`, development 1.3.27. Import `Type` and the `Static` type

from `typebox`. Use `Object`, `String`, `Literal`, `Array`, and `Number` to describe tool input and output. Schemas are module constants; there is no resource lifecycle. Tests use `Value.Check` to validate returned JSON. The empty input schema rejects additional properties.

Node standard library. `node:fs/readFileSync` reads the bundled asset once per factory invocation, using a module-relative URL; `node:crypto/createHash` verifies SHA-256. Reads are synchronous, small, and can throw at load time. `URL` is the standard global. No filesystem writes occur in application code.

Development only. TypeScript runs `tsc --noEmit` with strict NodeNext checking. Tests use `node:test`, `node:assert/strict`, `node:fs/promises`, `node:path`, `node:os`, `node:url`, and `node:child_process` for isolated SDK/CLI checks. Pi's public SDK supplies these test services: `DefaultResourceLoader`, `createAgentSession`, `ModelRuntime`, `SessionManager`, and `SettingsManager`. Tests dispose sessions and remove temporary files.

Pi's package manager supplies peers; do not add the host to runtime `dependencies`. `npm ci --ignore-scripts` prepares development. The lockfile pins transitive development dependencies. No imported dependency requires concurrency coordination in this package.

5 Repository and file design

`index.ts` Native default-exported factory, Pi event registration, command, flag, and transient state. Imports both helpers and Pi types.

`src/profile.ts` Prompt loader, fingerprint, TypeBox schemas, profile-data type, metadata builder, and tool factory. Imports Node fs/crypto, TypeBox, and Pi tool types. Used only by the adapter and tests.

`src/armor.ts` Pure text extraction, marker scanning, score and status types, and status formatting. Contains the inherited bilingual marker lists and non-refusal tails. No imports, I/O, or mutable globals.

`test/extension.test.ts` Actual Pi loader/runner integration, command/tool/prompt/session-state tests, asset integrity, and pure-score tests.

`test/smoke.test.ts` Real offline Pi CLI loading through a package directory and explicit entry-point flag discovery; isolated agent directory.

Metadata and instructions. Update the package manifest to version 0.3.0. Add a lockfile, strict TypeScript configuration, and Git ignore/attribute files. Rewrite the English README. Remove the old JavaScript adapter, browser client, Cordis declaration, Harness note, four installers, and unused sponsor image.

Unchanged. `LICENSE` and `prompts/infinite-gen-2.md`. Design documents and the one module note are reviewer artifacts, not runtime imports. Change evidence is recorded under the workspace root `docs/evidence/` and mirrored into this repository for the Git handoff.

6 Configuration and secrets

`--infinite-gen2-off` is a boolean flag, default false, non-secret. Read it during `session_start`; true creates a disabled initial state. `/infinite-gen2` accepts empty input, `status`, `on`, or `off` after trimming and lowercasing. Other values show usage and leave state unchanged. A session command overrides the startup

flag until the next session start. Disabling clears the verdict. Commands report through `ctx.ui.notify` when UI is available; otherwise use a displayed custom message with `triggerTurn: false`. No environment variable, secret, or persistent settings file is introduced. Pi's existing provider configuration is unchanged.

7 Module, class, and function contracts

This is one Pi extension module. The two helpers are implementation seams, not independent subsystems. No application classes are required.

`infiniteGen2Extension(pi): void` Default factory in `index.ts`. Load verified prompt before registering capabilities; initialize enabled idle state; register tool, flag, command, and lifecycle handlers. Side effects are registration only, with no spawned resources.

`publishArmorStatus(ctx): void` Factory-local helper. If `ctx.hasUI`, publish the formatted state under the unique key `infinite-gen2`. Otherwise do nothing.

`resetArmorStatus(ctx): void` Factory-local helper. Set enabled from the startup flag, clear result/outcome/running, and publish. Called on session start. Tree navigation clears results without changing enablement.

`loadPromptText(promptUrl?: URL): string` Read the default module-relative asset, or the supplied URL in focused integrity tests. Compute SHA-256 of raw bytes. If it differs from the frozen baseline, throw an Error; missing files retain Node's filesystem error. Decode UTF-8 only after matching. Never trim, normalize line endings, interpolate, translate, or rewrite text.

`buildProfile(promptText: string): InfiniteGen2Profile` Return a new object with legacy metadata and unchanged prompt. Version 0.3.0 is the adapter version; `promptVersion` remains 0.2.3. Produce fresh arrays so caller mutation cannot leak into subsequent results.

`createProfileTool(promptText): ToolDefinition` Return a directly exposed read-only tool, stable name `infinite_gen2_profile`, English label/description, empty object input schema, and concrete output schema. `execute()` creates a profile and returns text JSON, identical details, and identical structured content. No nested tools or model calls.

`extractResponseText(content: unknown): string` For an array, join only blocks with type text and a string text property with newlines. All other inputs/blocks contribute nothing. Thinking/tool/image content is never scored.

`findMarkerMatches(text, markers): string[]` Private scorer helper. Case-fold deterministically, scan non-overlapping occurrences, and exclude an occurrence if its next 12 characters include an inherited non-refusal tail. Return original marker labels, including repeated matches.

`scoreResponseText(text): ResponseScore` Compute refusal and fallback matches. Refusal takes precedence, fallback is second, otherwise pass; expose at most three combined marker labels. Empty text is not scored by the adapter.

`formatArmorStatus(state: ArmorState): string` English plain text, precedence disabled, busy, error/aborted, scored verdict, then enabled idle. Append "heuristic" to scored statuses; never claim model effectiveness.

All created/materially changed named symbols MUST have documentation comments with summary, behavior details, input records, and a return record in the workspace-mandated format.

8 Data models and internal contracts

`ResponseScore` contains `verdict: pass|refusal|fallback` and `words: string[]`. `ArmorState` contains `enabled: boolean`, `running: boolean`, `score: ResponseScore|null`, and `outcome: error|aborted|null`. Running tracks Pi activity even while injection is disabled, so mid-run enablement cannot claim idle. Each factory owns one state object; there is no cross-session store or module-global mutable state.

The profile output schema requires `name`, `displayName`, `version`, `promptVersion`, `baseline`, `lineage: string[]`, `strengthCalibration` with numeric `v4Obey`, `infiniteGen2`, and `infiniteGen1`, `stabilityFeatures: string[]`, `prompt: string`, and `promptSha256: string`. All other fields are strings. Keep original lineage, display name, calibration values 100/85/60, and feature list. These are inherited metadata, not measured scores. No database, migrations, retention, pagination, HTTP endpoints, or event-bus protocol is applicable.

The adapter calls helpers synchronously with immutable prompt text and ordinary value objects. Helpers do not mutate caller input. The profile tool remains registered while disabled; disabling affects instructions and monitoring, not asset inspection.

9 External SDK operations and runtime flow

`pi.on(before_agent_start, ...)` Use the event's structured prompt options. Mutate only `sections.infinite_gen2`: assign exact text when enabled, delete when disabled. Return no full-system override. Never change another section, tool list, or `forceSystemPrompt`. Pi supplies fresh normalized options on each run; assigning the same key is idempotent. An earlier forced full prompt remains the other extension's responsibility.

Session start/tree. Session start calls reset. Tree navigation clears running/score/outcome while retaining enabled. Shutdown removes the keyed status if UI exists. No attempt is made to reconstruct historical verdicts.

Run start. Set running true and clear prior score/outcome, even when disabled. Publish busy when enabled; disabled formatting remains off.

Message end. Ignore non-assistant messages and all messages while disabled. For assistant stop reason error/aborted, clear score and set outcome. Otherwise extract text and, if nonblank, replace score and clear outcome. Do not mutate messages or return replacements. Busy remains true throughout tool turns and retries.

Automatic compaction failure. On `session_compact_failed`, if enabled and running and reason is not manual, clear the score and record aborted when `event.aborted` is true, otherwise error. Pi can emit this after a completed response without another assistant message. An ensuing run start resets the outcome when Pi retries. Idle/manual failures do not change it.

Settlement. On `agent_settled`, set running false and publish. Do not queue messages or return continuation requests.

Commands. Normalize argument; update enabled and clear stale verdict on on/off while preserving running; status is read-only. Notify or send a non-triggering custom message. Invalid input reports usage without state changes. Mid-run disabling removes the section on the next run; it cannot retract instructions already sent.

The tool API uses `content`, `details`, `outputSchema`, and `structuredContent`, not the original Harness `output` adapter. The package manifest declares `pi.extensions: ["./index.ts"]`, `pi.prompts: []`, and no other resources, so the frozen prompt asset never becomes an accidental slash-command template. File allowlisting includes entry point, helper source, prompt, README, and license.

10 Failure behavior, observability, and security

There is no retry, backoff, timeout, external I/O, or long-lived resource to configure in production. Asset read/hash errors fail extension loading; Pi surfaces diagnostics. Marker false positives do not change assistant output, automatically re-prompt, or trigger side effects. Error/aborted responses do not receive pass status. Non-text responses retain the previous textual score within a run; a run with no textual result ends in enabled idle state.

The status key and command notifications are the only observability outputs. No raw conversation, credentials, or private project content is logged or written by the extension. Tests never use the real user's agent directory. The bundled prompt is untrusted instruction content that the user explicitly chose to install, not a source of runtime permissions. The extension does not patch Pi, disable approval gates, suppress errors, or alter credential handling. Another extension can replace the entire prompt later; document this limitation instead of trying to seize precedence. Package installation is explicit and uses Pi's own trust and dependency flow.

11 Testing design

Use one cohesive integration/unit test file and one CLI smoke boundary.

(i) Verify the literal baseline SHA against asset bytes; load from a changed or missing temporary file and expect deterministic rejection. Ensure the profile's prompt is byte-identical and its structured output passes its schema.

(ii) Load the real entry point via `DefaultResourceLoader`, with explicit path, isolated cwd/agent directory, no discovered extensions, skills, prompts, themes, or context files. Create an in-memory SDK session with an offline ModelRuntime and no refresh. Bind extensions, record runner errors, and dispose in test cleanup. No `session.prompt` call may contact a model; only the registered slash command is dispatched through it.

(iii) Use the actual runner's `emitBeforeAgentStart` to prove exact owned section insertion, no full override, other-section preservation, repeated-event idempotence, disabling, and startup-flag behavior. Exercise tool registration and execute the actual profile tool with the runner's tool context.

(iv) Table-test English/Chinese refusals, fallback, case-folding, exclusions, three-marker truncation, and malformed/mixed content. Test lifecycle busy, settlement, errors/abort, automatic compaction failure/cancellation after a completed response, mid-run enablement, tree resets, session resets, shutdown cleanup, headless behavior, invalid commands, and independent factory instances.

(v) Spawn the actual offline CLI with `--help`, isolated agent directory, no ambient resources, and `-e .`. Also install the local package into an isolated agent directory and require subsequent configured-package discovery. Require successful exit and presence of the extension flag. Check dry-run tar contents include the prompt and helpers, and exclude obsolete/browser/sponsor assets.

Strict typechecking covers application and tests. Use literal expected values or independent schema/hash checks rather than self-derived expectations.

12 Build, run, and handoff

Install: `pi install git:github.com/FidesTherese/dsh-infinite-gen-2`. Try locally: `pi -e ./index.ts` or `pi -e .`. Development: `npm ci --ignore-scripts`, then `npm test`, `npm run check`, `npm run smoke`, and `npm pack --dry-run`. There is no server, port, migration, or deployment. Remove using `pi remove` with the same package source. Use a new session or reload after code changes. No published-history rewrite is needed for rollback: reverting the migration commit restores the old source.

Compile this DTD, the specification, and module note with LuaLaTeX from the workspace root so its read-only `tem.sty` resolves. Preserve template class, typography, and layout. Remove auxiliary files after final passes. Create syntax-highlighted red/green change evidence after the final source diff stabilizes, compile with LuaLaTeX, and include its repository mirror in the requested commit. Review the complete diff, keep prompt/license blobs unchanged, commit on `master`, and push without force.

13 Implementation roadmap

Step 1 — Freeze contracts and expose failing checks

Prerequisites: inspected baseline, Pi declarations, and template. Create the specification, this DTD, package/check configuration, and test files. Replace manifest metadata and define explicit Pi resource discovery. Write failing loader/behavior tests before replacing application code. Verify the original entry point does not provide a Pi default factory. The failing tests are the acceptance contract; no new application API exists yet.

Step 2 — Port pure helpers and tool

Prerequisites: Step 1 assertions. Create `src/armor.ts` and `src/profile.ts` with the exact contracts above. Test the integrity boundary, score tables, profile schema, and fresh-object ownership. Preserve the prompt bytes. Run the focused test file; adapter tests remain red until Step 3. No unrelated prompt redesign or marker

expansion is permitted.

Step 3 — Wire native Pi lifecycle

Prerequisites: helper contracts. Create `index.ts`, then remove obsolete Harness runtime/install assets. Register the native default factory, owned section, tool, flag, command, and status lifecycle. Run `npm test`, `npm run check`, and `npm run smoke`; fix contract mismatches before proceeding. No persistence or additional provider is introduced.

Step 4 — Reconcile and publish the verified change

Prerequisites: stable source and passing checks. Rewrite the English README, create/update the one module note, reconcile verification in all design artifacts, inspect tar contents and reviewed diff, and produce compiled change evidence. Re-run affected checks, compare prompt/license to baseline, commit all repository changes, and push the existing branch. Record the remote commit hash. Deferred items remain out of scope; do not claim a live DeepSeek model test.

14 Implementation Instructions for Coding Agent

Implement the roadmap sequentially and do not skip prerequisites. Treat this DTD as the contract; do not silently replace dependencies, APIs, schemas, module boundaries, or algorithms. Run each specified verification when feasible and update tests with code. Preserve dependency direction and unrelated changes. Report unavailable APIs, incompatible versions, contradictions, and unmet acceptance criteria. When evidence disproves a contract, update this DTD before implementing a conflicting design. Keep explicitly deferred work deferred. Validation before handoff must check module, file, dependency, API, data-model, failure-flow, configuration, test, and roadmap coverage against the resulting code.